

IDENTIFICAR TRAMAS I2C –ANALIZADOR LOGICO

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen - En esta práctica se buscó identificar y comprender las diferentes partes que componen una comunicación I2C. Para ello, se realizó la comunicación entre una Raspberry Pi Pico 2W y una pantalla OLED SSD1306, utilizando un analizador lógico (Logic 2) conectado a las líneas SCL y SDA del bus para visualizar las señales generadas durante la transmisión. A partir de las capturas obtenidas se observaron las condiciones de inicio y finalización, la dirección del dispositivo y las respuestas ACK y NACK. De igual manera, se realizaron pruebas para observar su comportamiento frente a direcciones válidas e inválidas, el método de escaneo `i2c.scan()` para comprobar la dirección de la pantalla y finalmente diferentes funciones a aplicar en la OLED, para así relacionar los códigos hexadecimales observados con acciones como el encendido, apagado, ajuste de contraste, inversión de imagen y envío de datos.

Palabras clave - ACK/NACK, analizador lógico, bus I2C, MicroPython, Raspberry Pi Pico, SSD1306, trama I2C.

Abstract - This practical exercise aimed to identify and understand the various components of I2C communication. To achieve this, communication was established between a Raspberry Pi Pico 2W and an SSD1306 OLED display, using a logic analyzer (Logic 2) connected to the bus's SCL and SDA lines to visualize the signals generated during transmission. Analysis of the captured data revealed the start and stop conditions, the device address, and the ACK and NACK responses. Additionally, tests were conducted to observe behavior with valid and invalid addresses, the `i2c.scan()` method was used to verify the display's address, and various functions were executed on the OLED to correlate the observed hexadecimal codes with actions such as turning the display on or off, adjusting contrast, inverting the image, and sending data.

Keywords - ACK/NACK, I2C bus, I2C frame, logic analyzer, MicroPython, Raspberry Pi Pico, SSD1306.

I. INTRODUCCIÓN

El protocolo I2C (Inter-Integrated Circuit) es un bus de comunicación serial síncrono que permite la interacción entre un dispositivo maestro y uno o varios dispositivos esclavos mediante las líneas SCL, utilizada para la señal de reloj, y SDA, empleada para la transmisión de datos. Una transacción I2C

está conformada por diferentes elementos que permiten establecer y verificar la comunicación, entre ellos las condiciones de inicio (Start) y finalización (Stop), la dirección del dispositivo, el bit de lectura/escritura (R/W) y la respuesta de confirmación ACK o NACK.

En esta práctica se realizó el reconocimiento de tramas I2C utilizando una Raspberry Pi Pico 2W programada en MicroPython, una pantalla OLED SSD1306 y un analizador lógico. A partir de las señales capturadas en las líneas SCL y SDA, se identificaron los diferentes elementos de una transacción y se comparó el comportamiento del bus ante una dirección válida y una dirección incorrecta. Además, se empleó la función `i2c.scan()` para verificar la dirección del dispositivo conectado y se analizaron los códigos hexadecimales transmitidos para controlar diferentes funciones de la pantalla OLED.

II. OBJETIVOS

A. Objetivo General

Identificar la estructura de una transacción I2C (condición de Start, dirección de 7 bits más bit R/W, bit de ACK/NACK y condición de Stop) generada entre una Raspberry Pi Pico 2W y una pantalla OLED SSD1306, relacionando lo observado en la traza con el comportamiento esperado según la teoría del protocolo.

B. Objetivos Específicos

- Configurar el analizador lógico Logic 2 con el decodificador I2C habilitado y una tasa de muestreo adecuada para capturar correctamente las señales SCL y SDA.
- Comparar el comportamiento del bus I2C ante una dirección válida (ACK) y una dirección inválida (NACK), identificando qué elementos de la trama cambian y cuáles se mantienen.
- Verificar mediante la función `i2c.scan()` de MicroPython la dirección del dispositivo conectado al bus y contrastarla con la evidencia capturada por el analizador lógico.
- Analizar los códigos hexadecimales de control enviados a la pantalla OLED SSD1306 para distintas funciones (encendido, apagado, contraste, inversión y escritura de datos), relacionando cada código con la función que desempeña dentro de la comunicación con el dispositivo.

III. MARCO TEÓRICO

A. Protocolo I2C y estructura del bus

El protocolo I2C (Inter-Integrated Circuit) es un bus serial síncrono utilizado para la comunicación entre dispositivos mediante dos líneas:

SCL = Señal de reloj
SDA = Transmisión de datos.

Ambas líneas requieren resistencias de pull-up y pueden ser controladas por los dispositivos conectados al bus. En una comunicación convencional, el dispositivo maestro genera la señal de reloj e inicia las transacciones, mientras que el dispositivo esclavo responde a la dirección asignada [1], [4].

B. Condiciones Start y Stop

Una transacción I2C está delimitada por dos condiciones especiales generadas por el maestro:

- Start (S): transición de SDA de alto a bajo mientras SCL permanece en alto. Marca el inicio de la comunicación.
- Stop (P): transición de SDA de bajo a alto mientras SCL permanece en alto. Finaliza la transacción y libera el bus. [1].

Estas condiciones permiten delimitar cada transacción y facilitan su identificación mediante el analizador lógico. [1], [4].

C. Dirección del dispositivo y bit R/W

Después de la condición Start, el maestro transmite los 7 bits de dirección del dispositivo seguidos del bit R/W, el cual indica el sentido de la transferencia [1].

R/W = 0 operación de escritura
R/W = 1 operación de lectura

El octeto transmitido se obtiene mediante:

$$\text{Octeto} = (\text{Dirección} \ll 1) | \text{R/W}$$

Para la dirección de la pantalla OLED:

$$\text{x3C} = 0111100$$

Para una operación de escritura:

$$(\text{0x3C} \ll 1) | 0 = \text{0x78}$$

Por tanto, aunque la dirección del dispositivo sea 0x3C, el byte observado en la trama durante una escritura será 0x78.

D. Respuesta ACK y NACK

Tras cada octeto se transmite un noveno pulso de reloj destinado a la confirmación. Durante este pulso, el transmisor libera SDA y el receptor determina su estado. En un ACK, el receptor fuerza SDA a nivel bajo (0); en un NACK, SDA permanece en alto (1) porque ningún dispositivo la fuerza a bajo [1].

E. Frecuencia de SCL y frecuencia de muestreo

La velocidad de transmisión del bus está determinada por la frecuencia de la señal SCL. [2]. El período correspondiente a un ciclo de reloj se obtiene mediante:

$$T_{SCL} = \frac{1}{F_{SCL}}$$

Donde T_{SCL} es el período de la señal de reloj y F_{SCL} es su frecuencia. Por ejemplo, para una frecuencia de 100 kHz:

$$T_{SCL} = \frac{1}{100 \text{ kHz}} = 10 \mu\text{s}$$

F. Analizador lógico

Instrumento de medición electrónica diseñado para capturar, almacenar y visualizar señales digitales procedentes de múltiples canales de forma simultánea. En una comunicación I2C, permite observar las señales SCL y SDA y utilizar un decodificador para interpretar la información transmitida, facilitando la identificación de las condiciones Start y Stop, la dirección, el bit R/W y las respuestas ACK/NACK [6].

G. Función i2c.scan()

MicroPython ofrece el método `i2c.scan()`, que automatiza la búsqueda de dispositivos en el bus. Para las direcciones de 7 bits utilizables, 0x08 a 0x77, el microcontrolador genera una secuencia de sondeo que incluye Start, el octeto de dirección con R/W = 0, el bit ACK/NACK y Stop, y registra las direcciones que responden con ACK [2].

H. Byte de control y protocolo del controlador SSD1306

El controlador SSD1306 utiliza, sobre I2C, un byte de control enviado antes de los comandos o datos. En este byte son relevantes los bits Co (Continuation) y D/C# (Data/Command), que permiten distinguir entre comandos de configuración y datos destinados a la memoria GDDRAM. [5].

En la comunicación I2C con el SSD1306, el byte de control permite distinguir entre comandos y datos gráficos. De forma habitual, 0x80 se utiliza para indicar comandos y 0x40 para indicar datos destinados a la memoria GDDRAM [5].

IV. RESULTADOS EXPERIMENTALES

I. Reconocimiento de tramas I2C: ACK y NACK

En esta primera parte de la práctica se realizó el montaje de una comunicación I2C entre la Raspberry Pi Pico 2W y la pantalla OLED SSD1306, con el propósito de observar mediante el analizador lógico la estructura de una trama y reconocer las respuestas ACK y NACK.

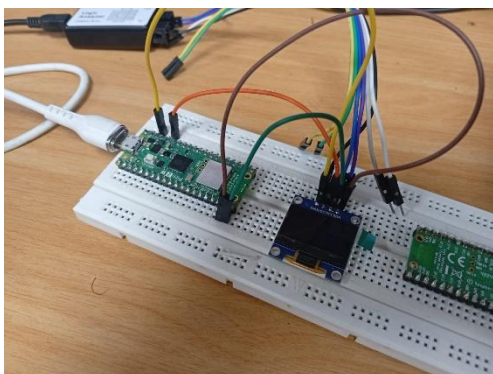


Figure 1. Montaje

La Raspberry Pi Pico 2W se configuró como dispositivo maestro y la pantalla OLED como dispositivo esclavo, utilizando las líneas SDA y SCL para la transmisión de datos y la señal de reloj, respectivamente.

A. Conexiones

Se estableció la conexión física entre los tres elementos del sistema: la Raspberry Pi Pico 2W (maestro I2C), la pantalla OLED SSD1306 (esclavo, dirección 0x3C) y el analizador lógico (instrumento de medición).

OLED

- SCL = GP15
- SDA = GP14
- VCC = 3.3 V
- GND = GND pico

Analizador lógico

- CH0 = SCL
- CH1 = SDA
- GND = GND pico

El bus I2C se configuró con una frecuencia nominal de 100 kHz mediante el parámetro freq=100000 del programa utilizado. El analizador lógico se configuró con una frecuencia de muestreo de 1 MS/s. Por tanto, la relación entre la frecuencia de muestreo y la frecuencia nominal del bus fue:

Relación de muestreo

$$N = \frac{1 \text{ MS/s}}{100 \text{ kHz}} = 10 \text{ muestras/periodo}$$

Esta configuración permitió obtener una cantidad suficiente de muestras para visualizar los pulsos de la señal SCL y los cambios de estado de SDA durante la transmisión. La conexión utilizada corresponde a la configuración indicada en la guía de laboratorio para el reconocimiento de las tramas I2C.

B. Prueba con dirección correcta (ACK esperado)

Se ejecutó el script OLED_ADDR_test.py con ADDR = 0x3C, el cual envía una transacción de sondeo formada por Start, el octeto de dirección, el bit ACK/NACK y Stop. Debido a que la OLED está presente en 0x3C, se esperaba una respuesta ACK. Para ver el código diríjase al **ANEXO A**

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Probing 0x3c ...
ACK de 0x3C
```

Figure 2.Mensaje Thonny ACK

La consola de Thonny confirma que el programa realizó el sondeo de la dirección 0x3C y recibió una respuesta ACK. El resultado es coherente con la presencia de la pantalla OLED en dicha dirección.

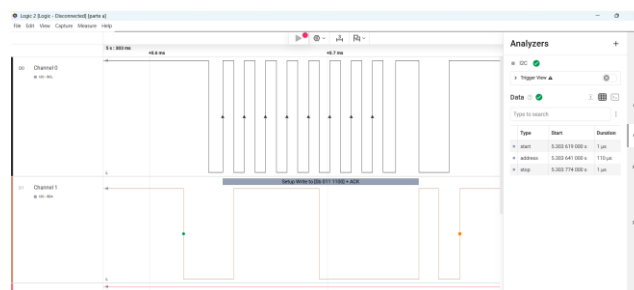


Figure 3.Captura ACK

Previo a la condición de Start, se observa en la captura que ambas líneas, SCL y SDA, permanecen en nivel alto, correspondiente al estado de reposo del bus.

En la traza se identifica la secuencia:

Start + dirección (0111100) + R/W = 0 + ACK + Stop.

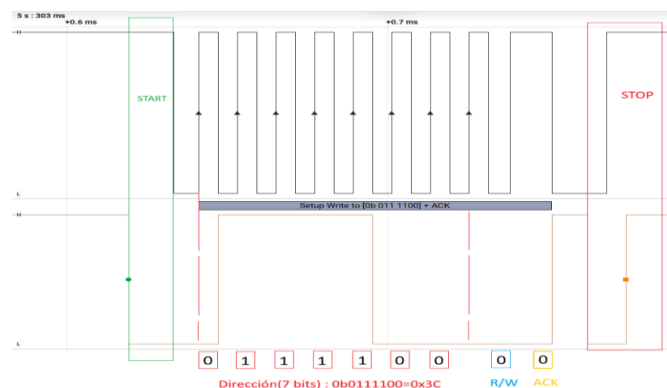


Figure 4.Secuencia completa ACK

Como la operación realizada corresponde a escritura, el bit R/W toma el valor 0. Por tanto, el octeto transmitido por el maestro es:

$$(0x3C \ll 1) | 0 = 01111000 = 0x78$$

El octeto completo transmitido es 0x78. El bit de ACK se observa en SDA como una caída a nivel bajo justo después de los 8 pulsos de dirección, confirmando que el esclavo reconoció su dirección.

C. Medición de la frecuencia de SCL

A partir de la captura de la trama ACK se midió una duración aproximada de 110 μ s para los 9 períodos de SCL correspondientes a los 8 bits del octeto de dirección y el bit de confirmación.

El período promedio de SCL se calculó mediante:

$$T_{medido} = \frac{110 \mu s}{9} = 12,22 \mu s$$

La frecuencia medida se obtuvo como:

$$F_{SCL} = \frac{1}{12,22 \mu s} = 81,8 \text{ kHz}$$

Al comparar este valor con la frecuencia nominal configurada de 100 kHz, se obtiene un error relativo de:

$$\%E = \frac{|81,8 - 100|}{100} \times 100 = 18,2\%$$

Por tanto, la frecuencia observada en la captura fue aproximadamente 81,8 kHz, inferior a los 100 kHz establecidos en la configuración del programa. Para una frecuencia ideal de 100 kHz, el período de cada ciclo sería de 10 μ s y los 9 ciclos ocupando aproximadamente 90 μ s.

La duración medida de 110 μ s representa aproximadamente 20 μ s adicionales respecto a este valor ideal. Con la frecuencia medida de 81,8 kHz y una tasa de muestreo de 1 MS/s, se obtienen aproximadamente:

$$\frac{1 \text{ MS/s}}{81,8 \text{ kHz}} = 12,2 \text{ muestras por período de SCL.}$$

Esto permitió distinguir los ciclos de SCL y observar los cambios de estado de SDA asociados a la transmisión

D. Prueba con dirección incorrecta (NACK esperado)

Posteriormente, se modificó la dirección utilizada en el programa a ADDR = 0x3D, con el propósito de transmitir una dirección diferente a la de la pantalla conectada y observar la respuesta NACK. Diríjase al **ANEXO B**

La dirección de 7 bits corresponde a: $0x3D = 0111101$

Al tratarse nuevamente de una operación de escritura, el bit R/W es 0. Por tanto, el octeto transmitido es:

$$(0x3D \ll 1) | 0 = 01111010 = 0x7A$$

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Probing 0x3D ...
NACK / no response: [Errno 19] ENODEV
>>> |
```

Figure 5. Mensaje Thonny NACK

La consola de Thonny confirma que el programa realizó el sondeo de la dirección 0x3D y no recibió reconocimiento del dispositivo. El resultado NACK es coherente con que la pantalla OLED está conectada en la dirección 0x3C.

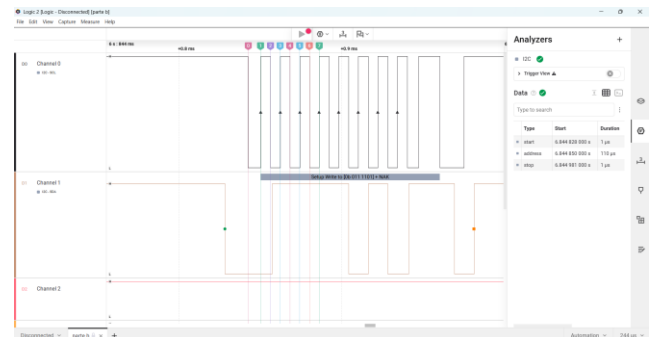


Figure 6. Captura NACK

En la traza se identifica la secuencia

Start + dirección (0111101) + R/W = 0 + NACK + Stop



Figure 7. Secuencia completa NACK

A diferencia de la captura ACK, durante el noveno pulso de SCL la línea SDA permanece en nivel alto, correspondiente a

NACK, porque ningún dispositivo del bus reconoce la dirección 0x3D. El maestro mantiene la misma estructura temporal de la transacción; la diferencia se encuentra en el contenido de la dirección transmitida y en la respuesta del receptor.

E. Comparación y análisis ACK/NACK

Elemento	Captura ACK (0x3C)	Captura NACK (0x3D)
Start detectado (SDA↓, SCL alto)	Sí	Sí
Octeto enviado	0x78	0x7A
Bit 9 (ACK=0 / NACK=1)	ACK (0)	NACK (1)
Stop detectado (SDA↑, SCL alto)	Sí	Sí
Duración de la fase de dirección	110 µs	110 µs
fSCL medida	82 kHz	82 kHz
Error relativo de fSCL respecto a 100 kHz	18,2 %	18,2 %

Tabla 1.Comparación entre la captura ACK y la captura NACK.

La igualdad de 110 µs en la fase de dirección produce una frecuencia medida de aproximadamente 81,8 kHz en ambas pruebas. Por tanto, el resultado temporal no depende de que el esclavo responda con ACK o NACK, sino de la generación de SCL por parte del maestro y de las condiciones de la captura.

El patrón de reloj SCL se mantiene esencialmente igual en ambas capturas: en los dos casos se transmiten ocho bits correspondientes a la dirección y al bit R/W, seguidos de un noveno pulso para la confirmación.

La diferencia se encuentra en el contenido del octeto y en el estado de SDA durante ese noveno pulso: para 0x3C se transmite 0x78 y el esclavo fuerza un 0 lógico (ACK), mientras que para 0x3D se transmite 0x7A y la línea permanece en 1 (NACK). Por tanto, cambia la dirección transmitida y la respuesta del receptor, mientras que la estructura temporal de la transacción se conserva.

II. Determinación automática de la dirección (i2c.scan())

Posteriormente, se realizó un escaneo automático del bus I2C mediante la función i2c.scan(). Para ello, se ejecutó el script scan_i2c_addr.py, encargado de recorrer las direcciones de 7 bits disponibles para el escaneo y registrar aquellas que responden con ACK. Para ver el código dirijase al **ANEXO C**.

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
0x3c
```

Figure 8.Consola de Thonny — el escaneo por software confirma la dirección 0x3c.

La consola de Thonny muestra el resultado obtenido mediante la función i2c.scan(), identificando la dirección 0x3C como

dispositivo presente en el bus. De esta manera, fue posible determinar automáticamente qué dispositivos se encontraban presentes en el bus sin necesidad de establecer previamente su dirección.

El rango utilizado comprendió desde 0x08 hasta 0x77, por lo que el número de direcciones sondeadas fue:

$$\text{Número de direcciones: } 0x77 - 0x08 + 1 = 112$$

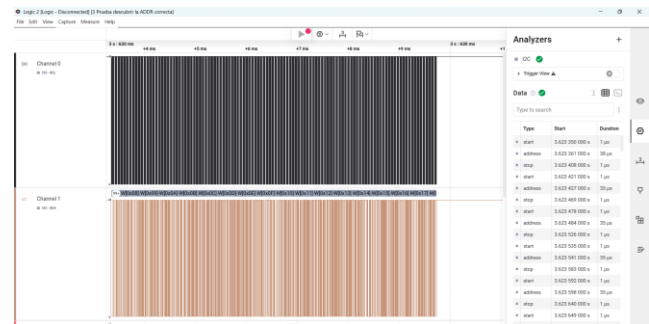


Figure 9.Vista general del escaneo

La captura muestra el proceso de exploración de las direcciones del bus I2C. El maestro genera sucesivos sondeos para determinar si existe un dispositivo que responda a cada dirección. La respuesta se establece mediante el bit de confirmación: un ACK indica reconocimiento de la dirección, mientras que un NACK indica que no se obtuvo reconocimiento.

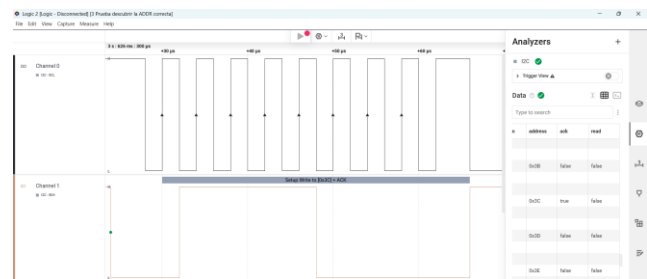


Figure 10.Resumen del analizador —0x3C ACK = true

En la tabla de resultados se observa que la dirección 0x3C presenta una respuesta ack = true, mientras que las demás direcciones mostradas, como 0x3B, 0x3D y 0x3E, presentan ack = false.

Esto permite identificar a 0x3C como la dirección que responde dentro del rango analizado. El resultado coincide con la dirección previamente utilizada en la prueba de ACK, en la cual la pantalla OLED respondió correctamente a la dirección 0x3C.

III. Análisis de códigos hexadecimales OLED SSD1306

Para esta parte de la práctica se ejecutó el programa OLED_demo_menu.py, mediante el cual se realizaron diferentes funciones de control sobre la pantalla OLED SSD1306. Durante la ejecución de cada opción, el analizador lógico permitió observar las tramas I2C generadas por la

Raspberry Pi Pico 2W e identificar los bytes transmitidos hacia la dirección del dispositivo. Para ver el código diríjase al **ANEXO D**

A partir de las capturas obtenidas se identificaron los bytes correspondientes a las funciones de apagado, encendido, ajuste de contraste, inversión de la imagen y transmisión de datos hacia la memoria GDDR4 de la pantalla. Para ver los resultados en la OLED diríjase al **ANEXO E**

1. Apagar la pantalla — 0xAE

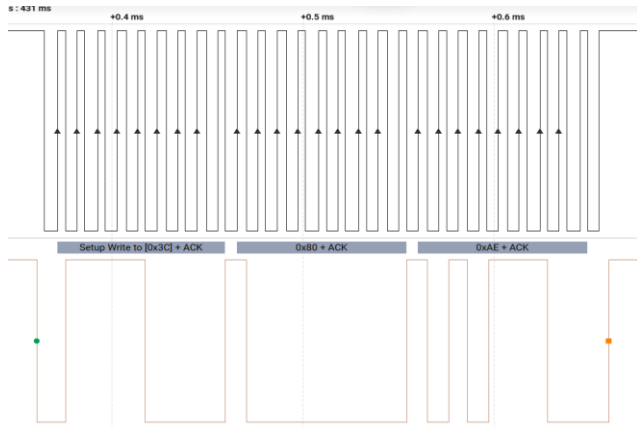


Figure 11. Trama I2C para apagar la pantalla: Write [0x3C] + ACK → 0x80 + ACK → 0xAE + ACK.

En la captura se observa una operación de escritura dirigida a la dirección I2C 0x3C, correspondiente a la pantalla OLED. Después de la dirección se presenta un ACK, lo que indica que el dispositivo reconoce la comunicación. A continuación, se transmite el byte de control 0x80, utilizado para indicar que el siguiente byte corresponde a un comando.

Posteriormente aparece el código 0xAE, seguido de su respectivo ACK. Este valor corresponde al comando Set Display OFF, por lo que la acción ejecutada desde el programa se relaciona directamente con el byte observado en el bus.

2. Encender la pantalla — 0xAF

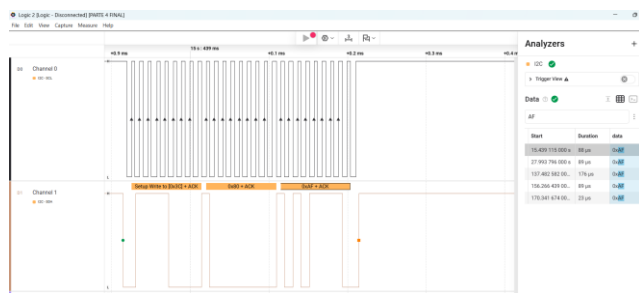


Figure 12. Trama I2C para encender la pantalla: Write [0x3C] + ACK → 0x80 + ACK → 0xAF + ACK.

La estructura de la transacción es equivalente a la observada en la Figura 11. Se inicia con la dirección 0x3C, seguida del byte de control 0x80 y del comando 0xAF. La diferencia se

encuentra en el último byte transmitido: 0xAF corresponde a la instrucción Set Display ON, utilizada para activar nuevamente la visualización de la pantalla. Cada byte transmitido recibe la correspondiente confirmación ACK.

3. Ajuste del contraste — 0x81 + valor

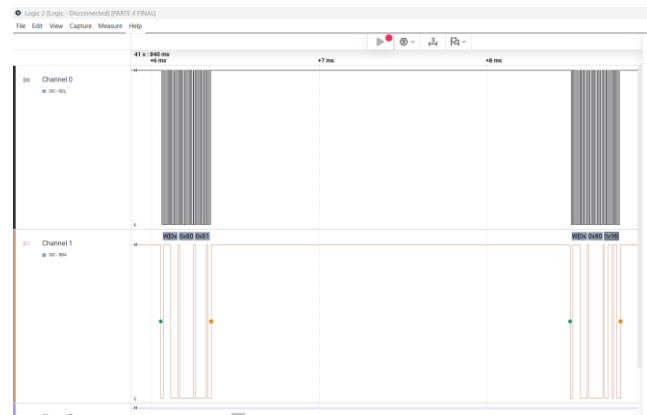


Figure 13. Transacciones I2C contraste mediante 0x81 y el valor 0x9B.

En esta captura se identifican dos transacciones consecutivas. En la primera se transmite el byte de control 0x80, seguido del comando 0x81, correspondiente a Set Contrast Control. En la segunda se transmite nuevamente el byte de control 0x80, seguido del valor 0x9B, equivalente a 155 en decimal.

Por tanto: $0x9B = 155_{10}$

A diferencia de los comandos de un solo byte, en este caso se observa la transmisión del comando de configuración y posteriormente el valor utilizado para establecer el nivel de contraste. Cada transacción presenta su correspondiente secuencia de comunicación y confirmaciones ACK.

4. Inversión de la imagen — 0xA7 / 0xA6



Figure 14. Trama I2C inversión de la imagen mediante el comando 0xA7.

La captura presenta la dirección de escritura 0x3C, seguida del byte de control 0x80 y del comando 0xA7. Este código corresponde a Inverse Display ON, por lo que permite activar la inversión de la imagen mostrada en la pantalla. Los bytes transmitidos reciben respuesta ACK, indicando que la comunicación fue reconocida.



5. Transmisión de texto y datos - 0x40



A diferencia de las instrucciones de control, en las que se utiliza el byte 0x80 para indicar la transmisión de comandos, el byte 0x40 permite establecer que los bytes siguientes corresponden a datos de visualización. Por esta razón, el analizador lógico muestra ráfagas de múltiples bytes consecutivos durante la transmisión de texto o gráficos.

A. Análisis de resultados

Opción del menú	Código(s)	Función (hoja de datos)
Apagar	0xAE	Set Display OFF
Encender	0xAF	Set Display ON
Contraste	0x81 + valor	Set Contrast Control (doble byte)
Invertir 1/0	0xA7 / 0xA6	Inverse / Normal Display

Tabla 2.Comandos identificados del controlador SSD1306.

De manera similar, 0xA7 y 0xA6 permiten modificar el modo de visualización, mientras que 0x40 identifica la transmisión de datos gráficos hacia la GDDR4M. Esto evidencia que el analizador lógico permite relacionar directamente las funciones ejecutadas por software con los bytes que circulan físicamente por el bus I2C

V. ANÁLISIS DE RESULTADOS

A. Análisis de las tramas ACK y NACK

Las capturas obtenidas mediante el analizador lógico permitieron identificar los principales elementos de una transmisión I2C: condición de inicio, dirección del dispositivo, bit R/W, bit de confirmación y condición de parada. Para la dirección válida del OLED, 0x3C, el maestro transmitió el octeto 0x78, correspondiente a la dirección de 7 bits desplazada un bit a la izquierda y con R/W = 0. En el noveno pulso de reloj se observó el ACK, representado por un nivel bajo en SDA, lo que confirma que el dispositivo reconoció el octeto transmitido.

Al realizar la prueba con la dirección 0x3D, el octeto transmitido fue 0x7A y durante el noveno pulso SDA permaneció en nivel alto, identificándose un NACK. La comparación entre ambas capturas demuestra que el analizador lógico permite diferenciar una dirección reconocida de una no reconocida mediante el estado de SDA durante el noveno ciclo de SCL. De esta manera, se comprobó experimentalmente la función del bit ACK/NACK dentro de la trama I2C.

B. Análisis de la frecuencia del bus I2C

Aunque el bus fue configurado en el programa con una frecuencia nominal de 100 kHz, la medición realizada con el analizador lógico presentó un período promedio de aproximadamente 12,22 μ s, equivalente a una frecuencia de 81,8 kHz. Esto representa una diferencia aproximada del 18,2 % respecto al valor configurado.

El resultado muestra que la frecuencia solicitada mediante software y la frecuencia efectivamente observada en el bus no necesariamente coinciden exactamente. Sin embargo, la señal mantuvo una periodicidad estable durante las capturas, permitiendo identificar correctamente los ciclos de reloj y realizar la decodificación de las tramas. Además, con una tasa de muestreo de 1 MS/s se obtuvieron aproximadamente 12

muestras por período de SCL, proporcionando suficiente resolución temporal para reconocer los cambios de SDA y SCL registrados.

C. Análisis del escaneo de direcciones

El uso de `i2c.scan()` permitió verificar mediante software qué dispositivos respondían en el bus. El escaneo comprendió el rango de direcciones I2C utilizado por la función y permitió identificar la dirección 0x3C como la única dirección que respondió positivamente en la prueba realizada. Las direcciones cercanas, como 0x3B, 0x3D y 0x3E, no presentaron respuesta.

Este resultado coincide con las pruebas realizadas directamente en el analizador lógico: la dirección 0x3C produjo un ACK, mientras que la dirección 0x3D utilizada en la prueba de NACK no fue reconocida. Por tanto, el escaneo permitió complementar la observación de las tramas, relacionando la identificación automática realizada por el programa con el comportamiento eléctrico observado en el bus.

D. Análisis de los comandos del SSD1306

Las capturas de las comunicaciones con el controlador SSD1306 permitieron relacionar las funciones ejecutadas desde el programa con los bytes transmitidos por el bus I2C. Se identificaron comandos como 0xAE y 0xAF para modificar el estado de la pantalla, 0x81 acompañado de un valor para establecer el contraste, y 0xA7 y 0xA6 para modificar el modo de visualización. También se identificó 0x40 asociado con la transmisión de datos gráficos hacia la memoria de visualización.

El análisis demuestra que las acciones realizadas sobre el OLED desde el programa se traducen en secuencias concretas de bytes que pueden ser observadas directamente mediante el analizador lógico. Esto permitió comprobar que la herramienta no solo facilita la visualización de las señales SCL y SDA, sino que también permite interpretar el contenido de las comunicaciones y relacionarlo con las funciones ejecutadas por el microcontrolador.

VI. CONCLUSIONES

1. Se identificaron los principales elementos que conforman una transacción del protocolo I2C mediante las capturas obtenidas con el analizador lógico. Reconociendo de esta manera las condiciones de inicio (Start) y finalización (Stop), la transmisión de la dirección de 7 bits, el bit de lectura/escritura (R/W) y el noveno ciclo correspondiente a la respuesta del dispositivo. Permitiendo así establecer la secuencia que sigue una comunicación I2C desde el inicio de la transmisión hasta su finalización.

2. Se reconoció experimentalmente la diferencia entre las respuestas ACK y NACK mediante el envío de dos direcciones diferentes. Para la dirección 0x3C, correspondiente a la pantalla OLED, se observó un ACK durante el noveno ciclo de reloj,

mientras que para la dirección 0x3D se obtuvo un NACK. Esto permitió comprobar que el estado de la línea SDA durante el noveno pulso de SCL permite determinar si la dirección transmitida fue reconocida o no por un dispositivo conectado al bus.

3. Se determinó, a partir de las mediciones realizadas sobre las capturas del analizador lógico, una frecuencia de SCL aproximada de 81,8 kHz, mientras que la frecuencia nominal configurada en la Raspberry Pi Pico 2W fue de 100 kHz. La diferencia entre ambos valores correspondió a un error aproximado del 18,2 %, lo que permitió caracterizar el comportamiento real de la señal de reloj en el bus físico y contrastar la velocidad teorizada por software frente a la obtenida experimentalmente en el hardware.

4. Se verificó la dirección de la pantalla OLED mediante el procedimiento de escaneo automático implementado con la función `i2c.scan()`. El programa evaluó las 112 direcciones del rango entre 0x08 y 0x77, identificando la dirección 0x3C como la que presentó respuesta ACK, validando de esta manera la dirección utilizada durante la prueba de reconocimiento de la trama y confirmar mediante software la dirección del dispositivo conectado al bus I2C.

5. Se relacionaron las acciones realizadas sobre la pantalla OLED con los códigos hexadecimales observados directamente en las tramas I2C. Se identificaron los comandos 0xAE y 0xAF para el apagado y encendido de la pantalla, 0x81 junto con su valor de configuración para el ajuste del contraste, 0xA7 y 0xA6 para la inversión y restauración de la imagen, y 0x40 para la transmisión de datos destinados a la GDDR4M. Esta correspondencia permitió interpretar cómo las instrucciones generadas por el programa se representan como bytes dentro de la comunicación I2C y cómo estos son utilizados para controlar las diferentes funciones del SSD1306.

VII. REFERENCIAS

[1] Texas Instruments, "I2C Protocol," Application Note, 2026. Documento técnico de TI

[2] MicroPython, "class I2C – a two-wire serial protocol," MicroPython Documentation, 2026. [Online]. Available: <https://docs.micropython.org/en/latest/library/machine.I2C.html>

[3] Raspberry Pi Ltd., Raspberry Pi Pico 2 Datasheet, 2026. Datasheet oficial Raspberry Pi Pico 2

[4] AMD, "Protocol for Address and Data Transfer," AXI IIC Bus Interface Product Guide, 2024. Documentación AMD sobre I2C

[5] Solomon Systech, SSD1306: 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller, Rev.1.5.

[6] Saleae, “Logic Analyzers,” Saleae Documentation, 2026.
[En línea]. Disponible en: <https://support.saleae.com/user-guide/logic-analyzers>

VIII. ANEXOS

ANEXO A OLED_ADDR_test.py ACK

```
1 # OLED_ADDR_test.py
2 import machine, time
3
4 i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14), freq=100000)
5 ADDR = 0x3C
6
7 time.sleep_ms(1000) # tiempo para armar el analizador
8
9 print("Probing 0x3c ...")
10 try:
11     # (addr, buffer, stop) -> sin keywords
12     i2c.writeto(ADDR, b"", True) # START, 0x3C(W), ACK, STOP
13     print("ACK de 0x3C")
14 except OSError as e:
15     print("NACK / no responde:", e)
16     # Si tu firmware no permite buffer vacío, descomenta la
17     # siguiente línea:
18     # i2c.writeto(ADDR, b"\x00", True) # ver s 0x3C(W), ACK, 0x00,
19     # ACK, STOP
```

Consola

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
Probing 0x3c ...
ACK de 0x3c
>>>
```

Figure 17. Código OLED_ADDR_test.py ACK

ANEXO B. OLED_ADDR_test.py NACK

```
1 # OLED_ADDR_test.py
2 import machine, time
3
4 i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14), freq=100000)
5 ADDR = 0x3D
6
7 time.sleep_ms(1000) # tiempo para armar el analizador
8
9 print("Probing 0x3D ...")
10 try:
11     # (addr, buffer, stop) -> sin keywords
12     i2c.writeto(ADDR, b"", True) # START, 0x3C(W), ACK, STOP
13     print("ACK de 0x3D")
14 except OSError as e:
15     print("NACK / no responde:", e)
16     # Si tu firmware no permite buffer vacío, descomenta la
17     # siguiente línea:
18     # i2c.writeto(ADDR, b"\x00", True) # ver s 0x3C(W), ACK, 0x00,
19     # ACK, STOP
```

Consola

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
Probing 0x3D ...
NACK / no responde: [Errno 19] ENODEV
>>> |
```

Figure 18. Código OLED_ADDR_test.py NACK

ANEXO C. scan_i2c_addr.py

```
1 # Digital Communication UPMG
2 # jose.rugeles@unimilitar.edu.co
3 # SCAN I2C ADDR - Raspberry Pi Pico
4
5 import machine
6
7 # Create I2C object
8 i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14))
9
10 # Print out any addresses found
11 devices = i2c.scan()
12
13 if devices:
14     for d in devices:
15         print(hex(d))
```

Consola

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
0x3c
>>>
```

Figure 19. Código scan_i2c_addr.py

ANEXO D. OLED_demo_menu.py

```
37 addrs = i2c.scan()
38 print("I2C scan:", [hex(a) for a in addrs])
39 if not addrs:
40     raise SystemExit("No hay dispositivos I2C. Revisa SCL=GP15, SDA=GP14, 3V3 y GND, y pull-ups en SDA/SCL.")
41 ADDR = 0x3C if 0x3C in addrs else (0x3D if 0x3D in addrs else addrs[0])
42 print("Usando OLED en addr:", hex(ADDR))
43
44 # ----- Instanciar OLED usando la subclase (chunked) -----
45 # Intento primero 128x32 y si falla pruebo 128x64.
46 try:
47     oled = ChunkedSSD1306_I2C(128, 32, i2c, addr=ADDR)
48 except Exception:
49     oled = ChunkedSSD1306_I2C(128, 64, i2c, addr=ADDR)
50
51 # ----- Helpers RAW (útiles para el analizador) -----
52 def send_cmd_raw(cmd):
53     # ...
```

Consola

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
I2C scan: ['0x3c']
Usando OLED en addr: 0x3c
-> Texto enviado (ver ráfagas 0x40 + datos)

=== MENU OLED I2C (freq=100000 Hz, 128x32) ===
1. Apagar (0xAE)
2. Encender (0xAF)
3. Contraste (0-255)
4. Invertir I/O
5. Limpiar
6. Texto demo
7. Animación breve
8. Enviar COMANDO RAW
9. Enviar DATO RAW
F. Cambiar frecuencia I2C
0. Salir
>
```

Figure 20. Código OLED_demo_menu.py

ANEXO E. Capturas comandos OLED.

E1. Encendido



Figure 21. Comando Encendido

E2. Apagado

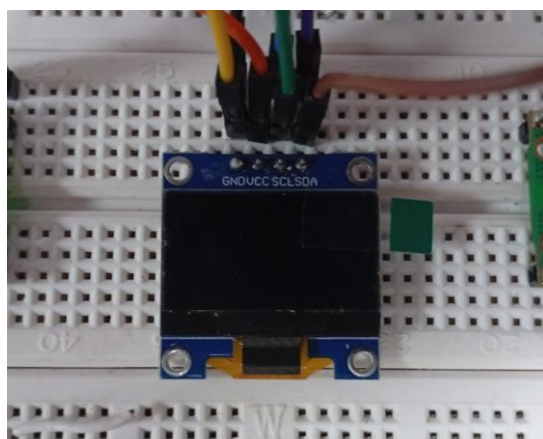


Figure 22.Comando Apagado}

E.3 Contraste



Figure 23.Comando Contraste (155)

E4. Invertir



Figure 24.Comando Invertir

E5. Texto /animación

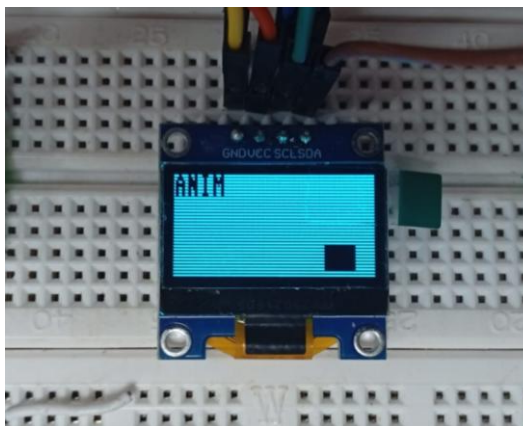


Figure 25.Comando Texto/Animación